

# Informe de Reflexión — Examen Final CI/CD

nnyez — 29 de julio de 2026 — <https://github.com/nnyez/inventario-app>

## 1. Justificación de la estrategia Canary

Elegir Canary en lugar de Blue-Green se justifica por cómo Kubernetes distribuye el tráfico: el Service balancea entre todos los pods que coinciden con su selector, sin importar la versión que ejecuten. Con cuatro réplicas estables (v1/blue) y una canary (v2/green), aproximadamente el 80 % del tráfico llega a la versión anterior y el 20 % a la nueva. Canary necesita solo cinco pods contra ocho de Blue-Green, lo que resulta más eficiente para una aplicación ligera como `inventario-app`. También permite revertir el cambio escalando el canary a cero réplicas, sin tocar el deployment estable.

## 2. Observación sobre persistencia de datos

Se agregó un producto vía POST (cuatro en total) y luego se eliminó un pod con `kubectl delete pod`. Kubernetes lo recreó de inmediato, pero el catálogo mostraba otra vez solo los tres productos iniciales. La base de datos es un archivo JSON local (`data/products.json`) dentro del contenedor. Al usar la imagen original, el pod nuevo arranca con los datos de fábrica. No se trata de un error, sino del comportamiento esperado cuando no hay un volumen persistente o una base de datos externa compartida.

## 3. Problemas reales encontrados

| Problema                    | Causa                                                     | Solución                           |
|-----------------------------|-----------------------------------------------------------|------------------------------------|
| Pipeline fallaba            | <code>trivy-action@0.28.0</code> no existe                | Cambiar a <code>@v0.36.0</code>    |
| Build fallaba por caché     | <code>cache-to=type=gha</code> incompatible               | Eliminar configuración de caché    |
| Error “denied: not allowed” | <code>GITHUB_TOKEN</code> sin <code>packages:write</code> | Agregar permissions al workflow    |
| Trivy CRITICALs             | <code>node:18-alpine</code> con CVE OpenSSL/tar           | Alpine 3.21, apk upgrade           |
| Pods no arrancaban          | Secret <code>inventario-app-secrets</code> no existía     | <code>kubectl create secret</code> |
| Sin balanceo en pruebas     | <code>port-forward</code> no distribuye tráfico           | Probar desde dentro del clúster    |

## 4. Métricas DORA

Los datos provienen de tres fuentes: `git log` (marca de tiempo del commit), `gh run list` (marca de tiempo del pipeline) y `kubectl describe deployment` (anotación `restartedAt`). Se consideraron los despliegues que efectivamente llegaron al clúster.

| Métrica                                          | Valor           | Nivel DORA          |
|--------------------------------------------------|-----------------|---------------------|
| <i>Lead Time</i> (commit → clúster)              | 16 seg – 41 min | <b>Élite</b> (<1 h) |
| Despliegue inicial: 3ab1521 04:23:30 → 05:04:29  | 41 min          |                     |
| Namespace + secrets: 112578a 05:06:48 → 05:07:04 | 16 seg          |                     |
| STARTUP_DELAY: 2d25c12 05:07:51 → ~05:08         | ~1 min          |                     |
| <i>Deployment Frequency</i>                      | 5 deploys/día   | <b>Alto</b>         |
| <i>Change Failure Rate</i>                       | 7/14 = 50 %     | <b>Bajo</b> (>30 %) |

## 5. Reflexión

El 50 % de tasa de fallo se explica por la etapa de montaje del proyecto. Siete de las catorce ejecuciones del pipeline fallaron por problemas de autenticación en GitHub Container Registry, una versión incorrecta de Trivy Action, conflictos con la caché de Docker y vulnerabilidades CRITICAL en la imagen base. Todos esos fallos ocurrieron en la primera hora de trabajo; una vez resueltos, los despliegues posteriores fueron limpios. En un entorno de producción consolidado esta tasa debería estar por debajo del 15 %, pero en una práctica de laboratorio la cifra refleja el aprendizaje más que una deficiencia del proceso.

El lead time más bajo (16 segundos) se dio en cambios de configuración como la creación del namespace o el ajuste del STARTUP\_DELAY. Eso indica que el pipeline, cuando no encuentra obstáculos, entrega cambios casi en tiempo real. La frecuencia de cinco despliegues en un solo día es propia de un entorno de experimentación; en un proyecto real esa cadencia dependería del ciclo de revisión y del tamaño del equipo.

Las métricas DORA no son fines en sí mismas. El valor de este ejercicio estuvo en ver cómo prácticas concretas –separar el build en etapas, escanear vulnerabilidades antes de publicar, definir probes que respeten el tiempo de arranque de la aplicación, y mantener los secretos fuera del repositorio– impactan directamente en la estabilidad de los despliegues. La parte canary, aunque simple, mostró que es posible convivir dos versiones en el mismo clúster sin romper el servicio.